

Relaciones entre clases: agregación y composición

Esteban Álvarez

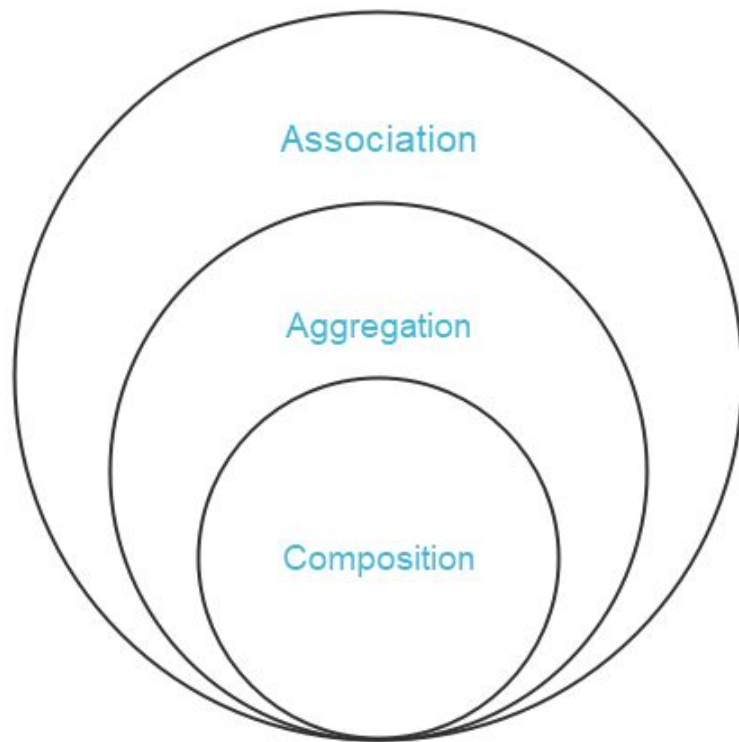
Introducción

- Hasta ahora hemos visto el funcionamiento de las clases de forma **individual**
- Pero en cualquier aplicación las clases se van a **relacionar entre ellas**

- Ejemplos:
 - Una persona tiene una o más mascotas
 - Un coche tiene un motor
 - Una bicicleta tiene dos ruedas
 - ...

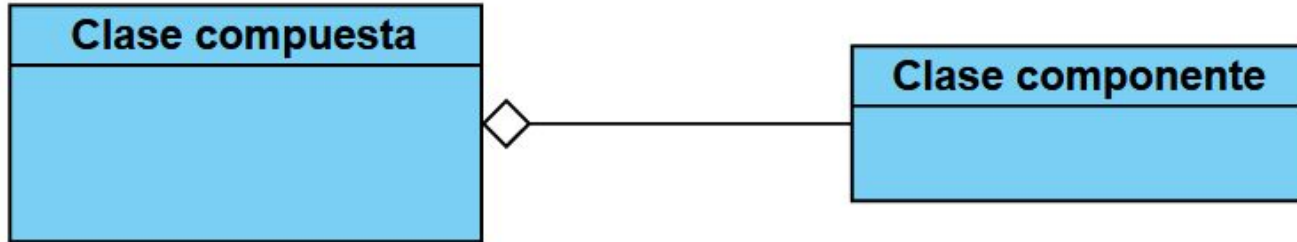
Introducción

- **Tipos** de relaciones en POO:
 - Agregación
 - Composición
- Es muy importante identificar esos tipos de relaciones y cómo se **implementan en java**



Agregación

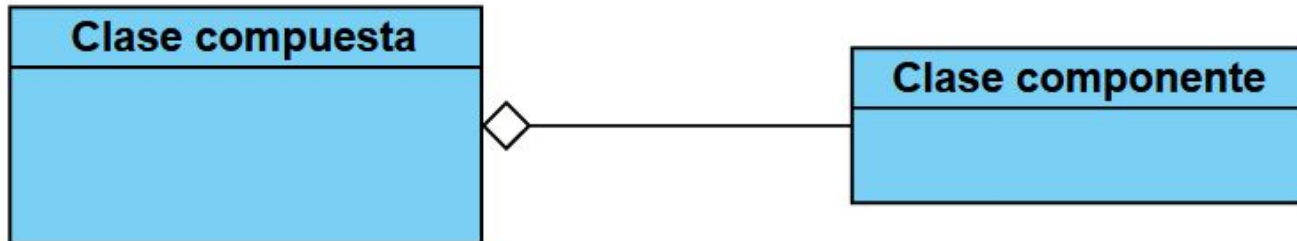
- La agregación es un tipo de asociación que indica que **una clase es parte de otra clase** (composición débil)
- Se representa en UML con un **diamante** en la clase compuesta



- La **destrucción** de la clase compuesta no implica la destrucción de los componentes

Agregación

- La **destrucción** de la clase compuesta no implica la destrucción de los componentes
- Ejemplos:
 - Clases Móvil y tarjeta SIM
 - Clases Ordenador y Webcam
 - Clases Coche y Radio
 - ...



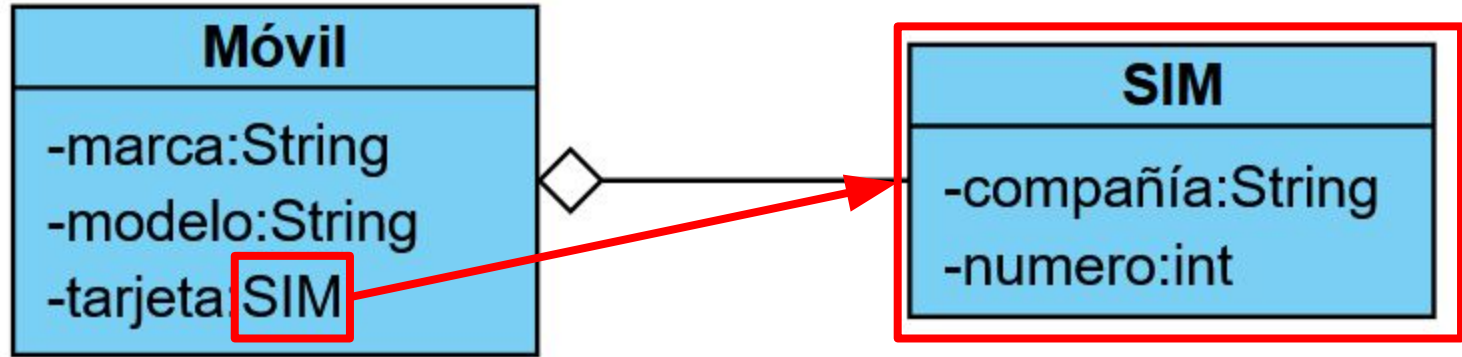
Agregación

Ejemplo:

- La clase Móvil tiene los atributos: marca, modelo y SIM
- La clase SIM es independiente y tiene los atributos: compañía y número

Agregación

Ejemplo:



- La clase Móvil **tiene una tarjeta SIM**
- La clase Móvil puede “vivir” sin la SIM: puede hacer todas las funciones menos las relacionadas con las llamadas y los mensajes
- La clase SIM también es independiente de la clase Móvil: se puede extraer e introducir en otro móvil
- Es una relación de **dependencia débil**

Agregación. Implementación en Java

- La clase Móvil **tiene una tarjeta SIM**

```
//Clase Movil.java
public class Movil {
    private String marca;
    private String modelo;
    private SIM tarjeta;
}
```

```
//Clase SIM.java
public class SIM {
    private String compania;
    private int numero;
}
```



Agregación. Implementación en Java

- La clase Móvil puede “vivir” sin la SIM: puede hacer todas las funciones menos las relacionadas con las llamadas y los mensajes
- La clase SIM también es independiente de la clase Móvil
- Eso significa que son **dos objetos independientes** en la aplicación

```
//Clase Principal.java  
Movil movil=new Movil("Samsung","Galaxy");  
SIM tarjetaSim=new SIM("Garrafón",600700800);
```

Agregación. Implementación en Java

- Para asociar la SIM creada con el Móvil, es necesario implementar un **método** en la clase Móvil
- El código java queda de la siguiente manera:

```
//Clase Movil.java
public void aniadirTarjeta(SIM nuevaSIM) {
    tarjeta=nuevaSIM;
}
```

```
//Clase Principal.java
Movil movil=new Movil("Samsung","Galaxy");
SIM tarjetaSim=new SIM("Garrafón",600700800);

movil.aniadirTarjeta(tarjetaSim);
```

Móvil
-marca:String -modelo:String -tarjeta:SIM
+aniadirTarjeta(nuevaSIM)

Agregación. Implementación en Java

- En el constructor de la clase Móvil hay que pararse a pensar:

Cuando se crea un móvil, ¿éste tiene tarjeta? **NO**

- Lo que se hace es inicializar a null la tarjeta
- Aunque también se podría llamar a su constructor por defecto

```
//Constructor de la clase Movil.java
public Movil(String marca, String modelo) {
    this.marca = marca;
    this.modelo = modelo;
    tarjeta=null;
}
```

Agregación. Getters y setters

- En la agregación (Tarjeta SIM) hay que recordar que se trata de **objetos independientes**
- Por tanto, los métodos get y set **pueden devolver y recibir** objetos de tipo SIM sin problemas:

```
//Método Añadir Tarjeta (Agregación).Equivalente a setTarjeta()  
public void aniadirTarjeta(SIM nuevaSIM) {  
    tarjeta=nuevaSIM;  
}  
//GetTarjeta().Al ser una agregación tiene sentido devolver el objeto SIM (tarjeta)  
public SIM getTarjeta() {  
    return tarjeta;  
}
```

Agregación. Getters y setters

- Uso del método get:

```
//Mostrar datos de la tarjeta. getTarjeta devuelve un objeto SIM  
System.out.println(movil2.getTarjeta());
```

- Uso del método “set”:

```
//Se añade la tarjeta creada a movil1  
movil1.aniadirTarjeta(tarjetaSim);
```

- Uso combinado:

```
//Añadimos la tarjeta de movil1 a movil2  
movil2.aniadirTarjeta(movil1.getTarjeta());
```

Agregación. Getters y setters

- Un método relacionado con añadirTarjeta puede ser el de **retirarTarjeta**
- Esto permite la **funcionalidad completa**: añadir o quitar una tarjeta a un móvil

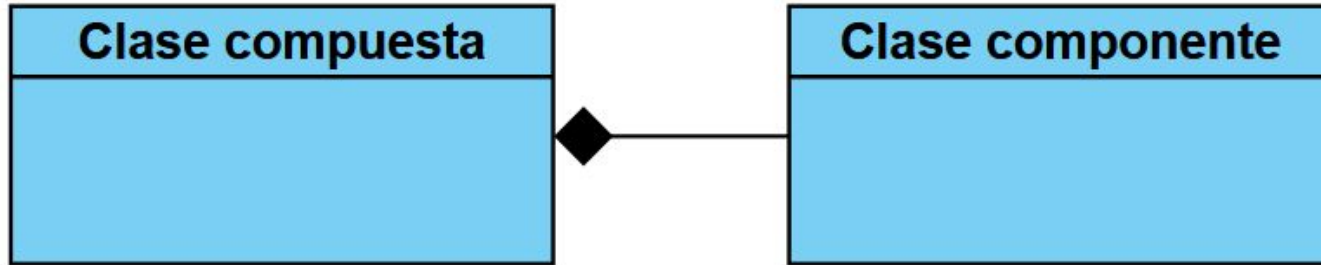
```
// Método Retirar Tarjeta (Agregación)
public void retirarTarjeta() {
    tarjeta = null;
}
```

Agregación. Resumen

- Se debe crear la clase Compuesta
 - Un atributo de la clase tiene que ser de la clase componente
 - En su constructor el atributo componente se inicializa a null
 - Se debe crear un método que permita añadir al componente
 - getters y setters pueden devolver y recibir objetos de la clase componente
- En la clase Principal:
 - Hay que crear dos objetos independientes: Compuesto y Componente
 - Se debe añadir el componente al compuesto

Composición

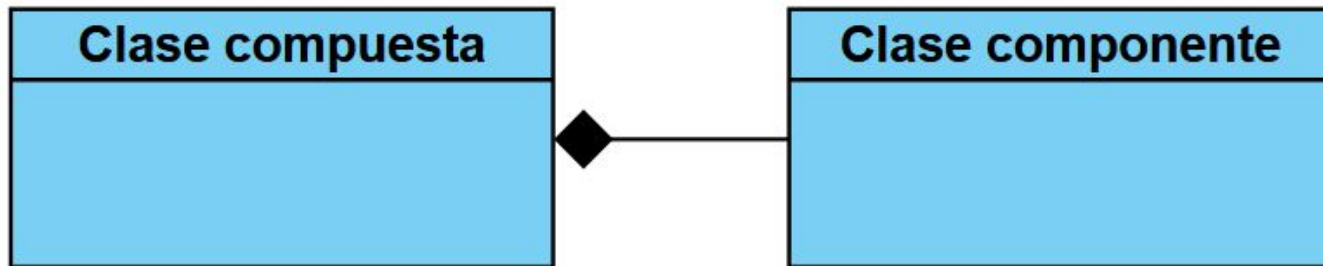
- La composición es una **agregación más fuerte**
- Un objeto compuesto no puede vivir sin uno de sus componentes
 - Una silla no puede existir sin sus patas
- Se representa en UML con un **diamante negro** en la clase compuesta



- La **destrucción** de la clase compuesta IMPLICA la destrucción de los componentes

Composición

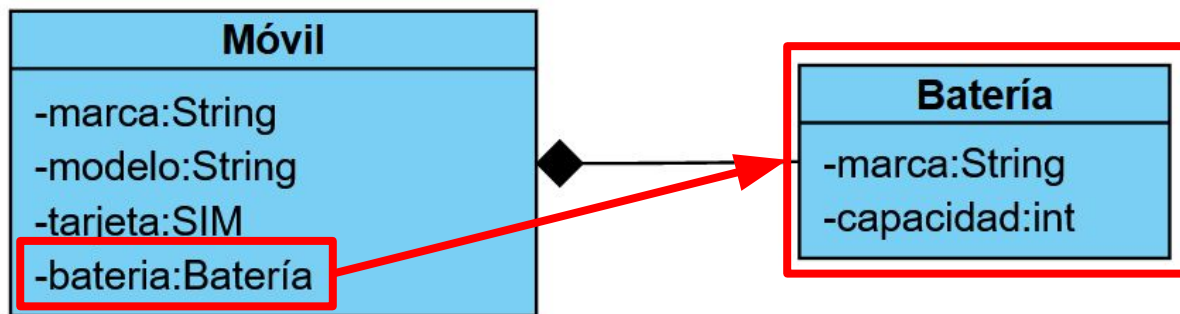
- La **destrucción** de la clase compuesta IMPLICA la destrucción de los componentes
- Ejemplos:
 - Persona, cabeza, cuerpo
 - Silla y patas
 - Avión y ala
 - ...



Composición

Ejemplo:

- La clase Móvil tiene los atributos: marca, modelo, SIM y **Batería**
- La clase Batería tiene los atributos: marca y capacidad
- Un móvil no puede funcionar si no tiene batería -> **Composición**



Composición. Implementación en Java

- La clase Móvil tiene una **Batería**

```
//Clase Móvil.java  
public class Movil {  
    private String marca;  
    private String modelo;  
    private SIM tarjeta;  
    private Bateria bateria;  
}
```

```
public class Bateria {  
    String marca;  
    int capacidad;  
}
```



Composición. Implementación en Java

- Un Móvil no puede funcionar si no tiene una **Batería**. Lo que implica que:
 - No hay dos objetos independientes, sólo hay UN Móvil que a su vez tiene UNA Batería
 - A la hora de crear un Móvil, hay que crearlo con Batería.

```
//Clase Principal.java
//Se crea un móvil con sus datos propios y los de la batería
Movil movil=new Movil("Samsung","Galaxy","Cutre cells",2800);
```

```
public Movil(String marca, String modelo,
              String marcaBateria, int capacidadBateria) {
    this.marca=marca;
    this.modelo=modelo;
    tarjeta=new SIM();
    //Creación de la batería con los datos que le pasan
    bateria=new Bateria(marcaBateria, capacidadBateria);
}
```

Composición. Implementación en Java

- Incluso para evitar que desde el Main se puedan crear objetos de tipo Batería, una posible alternativa es hacer que **Batería sea una clase interna de la clase Móvil**

```
//Clase Móvil.java
public class Movil {
    //1.ATRIBUTOS
    private String marca;
    private String modelo;
    private SIM tarjeta;
    private Bateria bateria;
```

```
//Clase Batería. Interna dentro de Móvil
private class Bateria{
    private String marca;
    private int capacidad;

    public Bateria(String marca, int capacidad) {
        this.marca = marca;
        this.capacidad = capacidad;
    }
    //Getters, setters, toString necesarios
}
```

Composición. Getters y setters

- En la composición (Batería) hay que recordar que el Main **no puede trabajar** con los objetos componente al pertenecer a una clase interna
- Por tanto, los métodos get y set **NO pueden devolver y recibir** objetos de tipo Batería:

```
/*
 * El método set debe hacer algo parecido a lo que hizo el constructor
 * con la Batería: Recibir sus datos y crear una nueva.
 */
public void setBateria(String marcaBateria, int capacidadBateria) {
    //Creación de la batería con los datos que le pasan
    //Igual que se hizo en el constructor
    bateria=new Bateria(marcaBateria, capacidadBateria);
}
/*
 * getBateria no puede devolver un objeto Batería.
 * Podría devolver, por ejemplo, un String con sus datos
 */
public String getBateria() {
    return bateria.toString();
}
```

Composición. Resumen

- Se debe crear la clase Compuesta
 - Un atributo de la clase tiene que ser de la clase componente
 - En su constructor se inicializan todos sus datos y se reciben datos para crear el objeto de la clase componente
 - Es recomendable que la clase componente sea interna a la clase compuesta
 - getters y setters no deben devolver o recibir objetos de la clase componente
- En la clase Principal:
 - Hay que crear un sólo objeto: Compuesto
 - En la llamada al constructor del objeto Compuesto, se pasan los datos para crear el objeto componente

Método toString

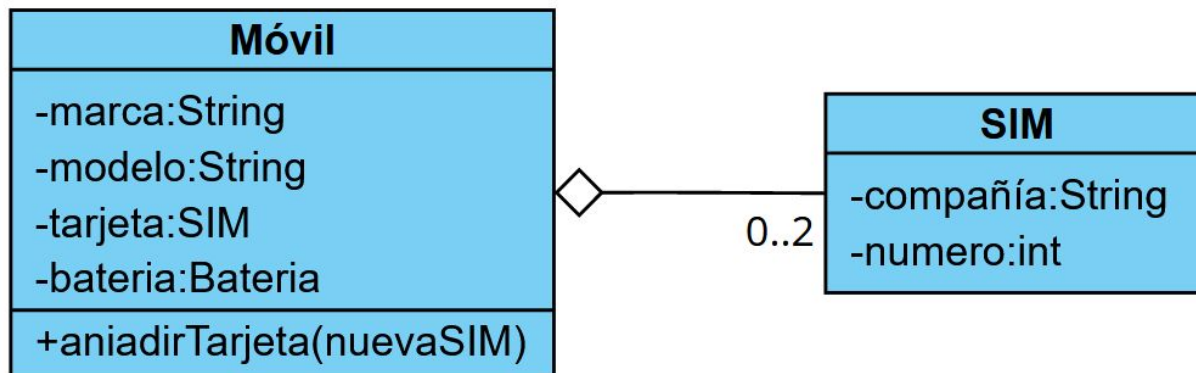
- Con el método toString no hay ningún problema
- Se genera automáticamente y se hace una **llamada en cadena** a los métodos toString de los objetos de las clases Componente

```
public String toString() {  
    return "Movil [marca=" + marca + ", modelo=" + modelo +  
        ", tarjeta=" + tarjeta + ", bateria=" + bateria + "];"  
}
```


Cardinalidades

- Al igual que en Bases de Datos, en Programación también podemos representar **cardinalidades**:

Un móvil puede tener cero, una o dos tarjetas SIM



Cardinalidades

- El **atributo** ahora no puede ser un único objeto de tipo SIM
- Debe ser una **colección** de objetos SIM, por ejemplo, un Array

```
public class Movil {  
    //1.ATRIBUTOS  
    private String marca;  
    private String modelo;  
    private SIM[] tarjeta;  
    private Bateria bateria;
```

- En la próxima Unidad se explicará su uso

Composición y Agregación. Resumen

- La **principal diferencia** a nivel de implementación entre agregación y composición viene dada por las características más restrictivas de la composición:

En la composición nuestro programa **debe**:

- **crear** el objeto componente al crear el objeto compuesto
- **eliminar** el objeto componente si se elimina el objeto compuesto

Ficheros .java

FIN!